

1.实现逻辑

区块链产生逻辑

区块链中的区块通常被实现为特定的结构体。每个区块包含了一些数据、一个时间戳、以及两个哈希值：一个是自身的哈希值，另一个是前一个区块的哈希值。

区块链中的区块通过这两个哈希值的连接来实现自动连接的。每个区块的哈希值都是由该区块的数据和前一个区块的哈希值计算得出的，因此每个区块的哈希值都依赖于前一个区块的哈希值。这种依赖关系确保了区块链中的每个区块都与其前一个区块链接在一起。

通过这种方式，如果任何一个区块的数据被篡改，那么它的哈希值将发生变化，这将破坏区块链的完整性。因为后续的区块的哈希值依赖于前一个区块的哈希值，因此如果前一个区块的数据被篡改，那么它的哈希值也会发生变化，从而导致整个区块链的哈希值序列被破坏。这使得区块链成为一种具有高度安全性和可靠性的数据结构

区块链创建与连接

通常情况下，在创建区块链时，你只需要确保按照正确的顺序创建区块，并确保每个区块的数据、时间戳、前一个区块的哈希值和本区块的哈希值是正确计算的。一旦你按照正确的方式创建了这些区块，并将它们存储在区块链中，它们就会自动连接。

具体来说，当你创建一个新的区块时，你会将前一个区块的哈希值作为当前区块的前一个哈希值存储在当前区块中。然后，你会计算当前区块的哈希值，这通常是通过当前区块的数据和前一个区块的哈希值进行哈希计算得到的。这种结构确保了每个区块都与其前一个区块连接起来。

因此，只要按照正确的方式创建和连接区块，区块链就会自动形成，每个区块都会正确连接到其前一个区块，从而形成一个完整的链条。

简而言之的逻辑前提

我们只需要按照时间戳，数据，前区块哈希和本块哈希来创建区块，区块会按照创建先后顺序自动连接。

2.实现建议代码

实现代码

```
package main

import (
    "bytes"
    "crypto/sha256"
    "encoding/binary"
    "fmt"
    "log"
    "time"
)

type Block struct {
    Timestamp int64
    Hash      []byte
    PrevHash  []byte
    Data      []byte
}
```

```

}

type Blockchain struct {
    Blocks []*Block
}

func (b *Block) SetHash() {
    information := bytes.Join([][]byte{ToHexInt(b.Timestamp), b.PrevHash,
b.Data}, []byte{})
    hash := sha256.Sum256(information)

    b.Hash = hash[:]
}

func ToHexInt(num int64) []byte {
    buff := new(bytes.Buffer)
    err := binary.Write(buff, binary.BigEndian, num)
    if err != nil {
        log.Panic(err)
    }
    return buff.Bytes()
}

func CreateBlock(prevhash, data []byte) *Block {
    block := Block{time.Now().Unix(), []byte{}, prevhash, data}
    block.SetHash()
    return &block
}

func GenesisBlock() *Block {
    genesiswords := "Hello, blockchain!"
    return CreateBlock([]byte{}, []byte(genesiswords))
}

func (bc *Blockchain) AddBlock(data string) {
    newBlock := CreateBlock(bc.Blocks[len(bc.Blocks)-1].Hash, []byte(data))
    bc.Blocks = append(bc.Blocks, newBlock)
}

func CreateBlockchain() *Blockchain {
    blockchain := Blockchain{}
    blockchain.Blocks = append(blockchain.Blocks, GenesisBlock())
    return &blockchain
}

func main() {
    blockchain := CreateBlockchain()
    time.Sleep(time.Second)
    blockchain.AddBlock("This is the first blockchain I have created!")
    time.Sleep(time.Second)
    blockchain.AddBlock("I want to add some basic information such as ma name
Always")
    time.Sleep(time.Second)
    blockchain.AddBlock("And my birthday is tomorrow 3.28,it is a good gift!")
    time.Sleep(time.Second)
}

```

```

    for _, block := range blockchain.Blocks {
        fmt.Printf("Timestamp: %d\n", block.Timestamp)
        fmt.Printf("hash: %x\n", block.Hash)
        fmt.Printf("Previous hash: %x\n", block.PrevHash)
        fmt.Printf("data: %s\n", block.Data)
    }
}

```

逐块解释

1.导入包

```

import (

    "bytes"
    //提供了操作字节切片的函数和方法。
    //在简易区块链中，字节切片用于存储和处理各种数据，比如区块的哈希值、交易数据等

    "crypto/sha256"
    //提供了 SHA-256 哈希算法的实现。

    "encoding/binary"
    //提供了将数据和字节表示进行转换的函数。
    //在简易区块链中，经常需要将整数、浮点数等数据类型转换为字节表示，以便进行哈希计算或者在网络
    中传输。

    "fmt"
    //格式化输入输出的函数

    "log"
    //提供了简单的日志功能。
    //在区块链应用中，日志是非常重要的，可以用于记录系统运行时的各种信息，比如错误信息、警告信息
    等。
    //log 包中的函数可以用于打印日志信息到标准输出或者其他指定的输出位置。

    "time"
    //提供了时间相关的功能。
    //在区块链中，时间戳是非常重要的，用于记录区块的创建时间。
    //time 包中的函数可以获取当前时间、格式化时间等，用于生成区块的时间戳。
)

```

2.区块结构体

```
type Block struct {
    Timestamp int64 //时间戳
    Hash      []byte //哈希值
    PrevHash  []byte //存储前一个区块的哈希值
    Data      []byte //存储的是数据
}
```

//在区块链中，各个区块的链接通常是通过哈希值来实现的。

//每个区块都包含了前一个区块的哈希值（PrevHash），这样就形成了一个链式结构。

//当一个新的区块被创建时，它的 PrevHash 字段会存储前一个区块的哈希值。

//在验证新区块时，可以通过计算前一个区块的哈希值并将其与新区块中的 PrevHash 进行比较，从而确保新区块的链接正确。

//这种链接方式使得区块链具有不可篡改性和完整性，因为一旦链中的一个区块被修改，它将导致其后续所有区块的哈希值发生变化，从而被轻松地检测到篡改。

3.头指针指向头区块

```
type Blockchain struct {
    Blocks []*Block //指向 Block 结构体的指针切片 指向第一个区块
}
```

//相当于链表里面的头指针，借助这个可以访问整个链

4.生成区块哈希值

```
func (b *Block) SetHash() {
    //b *Block go语言中对于结构体成员的访问都是用 . 无论是结构体还是结构体指针
    information := bytes.Join([][]byte{ToHexInt(b.Timestamp), b.PrevHash,
    b.Data}, []byte{})
    //bytes.Join() 函数将这些数据片段连接起来形成一个单独的字节片
    //将当前区块的时间戳 Timestamp、前一个区块的哈希值 PrevHash、以及当前区块的数据 Data 拼
    接成一个字节片（byte slice）
    //为下一步计算哈希值做准备
    hash := sha256.Sum256(information)
    //计算哈希值
    b.Hash = hash[:]
    // 最后一行将计算得到的哈希值赋值给当前区块的 Hash 属性。
    //由于 hash 的类型是 [32]byte，而 Hash 的类型是 []byte
    //Go 语言中的切片操作符，它表示从一个数组或切片中生成一个新的切片
    //[:] 表示从该数组的第一个元素到最后一个元素，因此 hash[:] 就是将整个数组转换为一个切片
}
```

5.将整数转换成字节切片

```
func ToHexInt(num int64) []byte {
    buff := new(bytes.Buffer)
    //bytes.Buffer变量可以理解为一个动态数组
    //new()函数用于创建一个新的变量，并返回该变量的指针。其工作方式类似于C语言中的malloc函数
    err := binary.Write(buff, binary.BigEndian, num)
    //相当于向开辟的缓冲区末尾追加数据
}
```

```

//buff: 是要写入数据的目标字节缓冲区。
//binary.BigEndian: 表示使用大端序（BigEndian）进行编码，这是一种将多字节数据在网络传输
或存储时常用的字节序。
//j就是按高位到低位将其读进字符串
//num: 是要写入的整数值。
if err != nil {
    log.Panic(err)
}
//检查了 binary.Write 是否返回了错误。
//如果有错误发生，binary.write 函数会返回一个非 nil 的错误，然后程序会通过
log.Panic(err) 来抛出异常并终止程序的执行。
return buff.Bytes()
}

// - `Write`: 将字节写入到缓冲中。
// - `WriteByte`: 将一个单字节写入到缓冲中。
// - `WriteRune`: 将一个 Unicode 字符写入到缓冲中。
// - `WriteString`: 将一个字符串写入到缓冲中。
// - `Read`: 从缓冲中读取字节。
// - `ReadByte`: 从缓冲中读取一个单字节。
// - `ReadBytes`: 从缓冲中读取字节直到遇到指定的分隔符。
// - `ReadString`: 从缓冲中读取字符串直到遇到指定的分隔符。

```

6. 创建一个新的区块

```

// 将前一个区块的哈希值和想要存储的数据导入 创建一个新的区块 并返回区块的地址
func CreateBlock(prevhash, data []byte) *Block {
    block := Block{time.Now().Unix(), []byte{}, prevhash, data}
    //将时间戳 传入的数据，前一个区块的哈希值导入，并且初始化一个字节切片来存储本区块的哈希值
    block.SetHash()
    //调用block结构体的SetHash方法来计算并设置当前区块的哈希值。
    return &block
}

```

7. 创建创世区块

```

func GenesisBlock() *Block {
    genesiswords := "Hello, blockchain!"
    return CreateBlock([]byte{}, []byte(genesiswords))
    //这个区块的特殊之处在于它没有前块哈希值
}

```

8. 添加区块到链中

```

func (bc *Blockchain) AddBlock(data string) {
    newBlock := CreateBlock(bc.Blocks[len(bc.Blocks)-1].Hash, []byte(data))
    bc.Blocks = append(bc.Blocks, newBlock)
}

```

9.创建区块链

```
func CreateBlockchain() *Blockchain {
    blockchain := Blockchain{}
    //创建了一个名为 blockchain 的 Blockchain 结构体实例。
    //这个结构体包含一个 Blocks 字段，用于存储区块链中的所有区块。
    blockchain.Blocks = append(blockchain.Blocks, GenesisBlock())
    //向 blockchain.Blocks 切片中追加了创世区块（Genesis Block）
    return &blockchain
}
```

10.主函数

```
func main() {
    blockchain := CreateBlockchain()
    time.Sleep(time.Second)
    blockchain.AddBlock("This is the first blockchain I have created!")
    time.Sleep(time.Second)
    blockchain.AddBlock("I want to add some basic information such as ma name Always")
    time.Sleep(time.Second)
    blockchain.AddBlock("And my birthday is tomorrow 3.28,it is a good gift!")
    time.Sleep(time.Second)

    for _, block := range blockchain.Blocks {
        //for _, block := range blockchain.Blocks 这行代码是 Go 语言中的循环语句，它被
        用来遍历 blockchain.Blocks 这个数组或切片中的所有元素。
        //在这个循环中，block 是每次迭代时代表数组中的一个元素，即一个区块。
        //在这种情况下，blockchain.Blocks 可能是一个存储了所有区块的切片。
        //这个切片中的每个元素都是一个区块，因此通过循环遍历这个切片，你可以逐个访问每个区块。
        //至于"自动连接"的部分，这是因为在每个区块的数据结构中，都包含了前一个区块的哈希值。
        //因此，通过遍历区块链中的每个区块，你可以逐个检查每个区块的前一个区块的哈希值是否与前一
        个区块的实际哈希值匹配，从而验证区块链的完整性。
        fmt.Printf("Timestamp: %d\n", block.Timestamp)
        fmt.Printf("hash: %x\n", block.Hash)
        fmt.Printf("Previous hash: %x\n", block.PrevHash)
        fmt.Printf("data: %s\n", block.Data)
    }
}
```

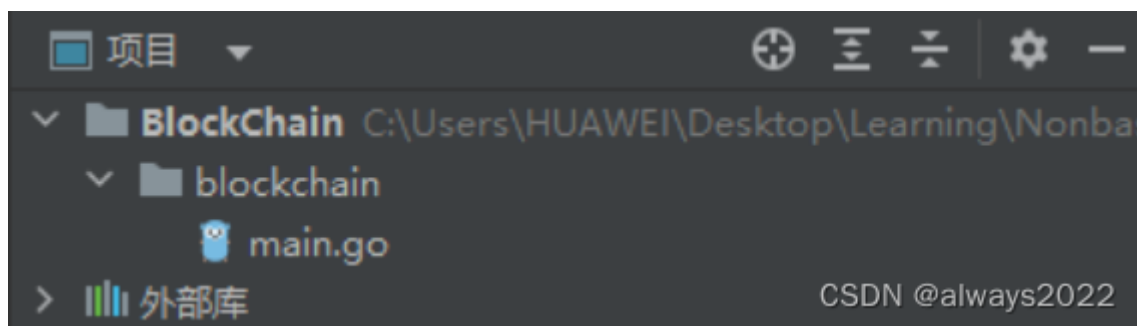
运行及结果

1. 项目创建及运行

创建项目

在terminal中输入

```
go mod init goblockchain
```



编辑

终端运行命令

1. 打开终端。
2. 使用 `cd` 命令导航到包含 `main.go` 文件的项目目录。
3. 运行以下命令编译并执行 `main.go` 文件：

```
go run main.go
```

2.运行结果

```
Timestamp: 1711517905
hash: 6c4d11fc38cf455b86da3566b62860a1cd72f27d2ebd4e3dc34ed3eb7471fc44
Previous hash:
data: Hello, blockchain!
Timestamp: 1711517906
hash: d663ed95c0cc3e6972b713a6a3eb5d6a45112ad7749c04bd8fff4c6d4cb863ec
Previous hash: 6c4d11fc38cf455b86da3566b62860a1cd72f27d2ebd4e3dc34ed3eb7471fc44
data: This is the first blockchain I have created!
Timestamp: 1711517907
hash: 0ca6aaa20e942124da26f15ce8c095fb853bdd949c084349fe53ef1fe18e481b
Previous hash: d663ed95c0cc3e6972b713a6a3eb5d6a45112ad7749c04bd8fff4c6d4cb863ec
data: I want to add some basic information such as ma name Always
Timestamp: 1711517908
hash: 5702b0dcd756408c0eed42926956fd15189135c32002d5089f8ca39956152aee
Previous hash: 0ca6aaa20e942124da26f15ce8c095fb853bdd949c084349fe53ef1fe18e481b
data: And my birthday is tomorrow 3.28,it is a good gift!
```